

Mémo — Authentification GitHub avec un token personnel

Pourquoi ? Depuis août 2021, GitHub n'accepte plus le mot de passe classique pour les opérations en ligne de commande (`git push`, `git pull`, etc.). Il faut utiliser un **Personal Access Token (PAT)** — une clé personnelle qui remplace le mot de passe.

Durée estimée : 5 minutes.

1. Créer le token sur GitHub

1.1 Accéder aux paramètres

1. Connectez-vous sur <https://github.com/>
2. Cliquez sur votre **photo de profil** (en haut à droite) → **Settings**.
3. Dans le menu de gauche, descendez tout en bas → **Developer settings**.
4. Cliquez sur **Personal access tokens** → **Tokens (classic)**.

💡 GitHub propose aussi des « Fine-grained tokens ». Pour ce cours, **les tokens classiques suffisent et sont plus simples**.

1.2 Générer un nouveau token

1. Cliquez sur **Generate new token** → **Generate new token (classic)**.
2. Confirmez votre mot de passe GitHub si demandé.
3. Remplissez le formulaire :

Champ	Valeur recommandée
Note	<code>Token cours Kanban</code> (un libellé pour vous souvenir)
Expiration	<code>90 days</code> (ou plus selon votre cours)
Scopes	Cochez uniquement <code>repo</code> (accès complet à vos dépôts)

4. Cliquez sur **Generate token** en bas de page.

1.3 Copier et sauvegarder le token

GitHub affiche votre token une **seule fois**, du type :

```
ghp_aBcD1234XYZ5678efGhIjKlMnOpQrStUvWxYz
```

⚠ **Important :** copiez-le immédiatement et collez-le dans un endroit sûr (gestionnaire de mots de passe, fichier local protégé). Si vous le perdez, il faudra en régénérer un nouveau — vous ne pourrez plus le retrouver.

2. Utiliser le token lors du `git push`

Il existe **deux méthodes**. Choisissez celle qui correspond à votre situation.

● Méthode A — Token directement dans l'URL (*recommandée*)

C'est la méthode **la plus fiable**, surtout si Git ne vous demande **ni username ni password** et échoue directement avec une erreur d'authentification.

A.1 Si c'est votre premier `git push`

Au lieu de l'URL classique, utilisez cette syntaxe en remplaçant les trois éléments en majuscules :

```
bash

git remote add origin https://VOTRE-USERNAME:VOTRE-TOKEN@github.com/VOTRE-USERNAME/m
git push -u origin main
```

Exemple concret :

```
bash

git remote add origin https://marie-dupont:ghp_aBcD1234XYZ5678efGhIjKlMnOpQrStUvWxYz
git push -u origin main
```

A.2 Si vous avez déjà configuré le remote sans token

Mettez à jour l'URL existante :

```
bash

git remote set-url origin https://VOTRE-USERNAME:VOTRE-TOKEN@github.com/VOTRE-USERNA
git push -u origin main
```

A.3 Vérifier que l'URL est bien enregistrée

```
bash

git remote -v
```

Vous devez voir votre URL avec le token.

⚠ **Sécurité** : avec cette méthode, le token est stocké **en clair** dans le fichier `.git/config` de votre projet. Ne partagez jamais ce dossier `.git/`, et ne committez **jamais** ce fichier. C'est acceptable pour un projet de cours, mais pas pour un projet professionnel sensible.

● Méthode B — Saisie interactive

À utiliser **uniquement** si Git vous demande **explicitement** le username et le password lors du push.

```
bash  
  
git push -u origin main
```

Git vous demande :

```
Username for 'https://github.com': votre-nom-utilisateur-github  
Password for 'https://votre-nom@github.com': [collez le token ici]
```

🔒 **Le token ne s'affiche pas** quand vous le collez (curseur immobile). C'est normal — appuyez sur **Entrée** après avoir collé.

3. Que faire si Git ne demande rien et échoue ?

C'est le cas typique où Windows ou macOS a déjà mémorisé d'anciens identifiants invalides dans le **gestionnaire de mots de passe système**. Git les utilise sans rien demander, et ça échoue.

3.1 Symptôme

```
remote: Support for password authentication was removed...  
fatal: Authentication failed for 'https://github.com/...'
```

→ Aucune demande d'username/password, échec direct.

3.2 Solution rapide (*recommandée*)

Utilisez la **méthode A** ci-dessus : mettez le token directement dans l'URL avec `git remote set-url`. Cela contourne le cache système.

3.3 Solution propre — vider le cache d'identifiants

Sur Windows :

1. Ouvrez le **Gestionnaire d'identification** (*Panneau de configuration → Comptes d'utilisateurs → Gestionnaire d'identification*).
2. Cliquez sur **Informations d'identification Windows**.
3. Cherchez les entrées `git:https://github.com`.
4. Cliquez dessus → **Supprimer**.
5. Relancez `git push`. Cette fois, Git redemandera username + token.

Sur macOS :

1. Ouvrez **Trousseau d'accès** (*Spotlight* → "*Trousseau*").
2. Cherchez `github.com`.
3. Supprimez les entrées trouvées.
4. Relancez `git push`.

Sur Linux :

```
bash
rm ~/.git-credentials
```

Puis relancez le push.

4. Mémoriser le token pour les pushes suivants

Une fois la première authentification réussie (méthode A ou B), activez le **credential helper** pour ne plus retaper le token :

Sur Windows :

```
bash
git config --global credential.helper manager
```

Sur macOS :

```
bash
git config --global credential.helper osxkeychain
```

Sur Linux :

```
bash
git config --global credential.helper store
```

5. Problèmes fréquents

Problème	Solution
<code>Authentication failed</code> sans demande d'username/password	Cache système avec anciens identifiants → utilisez la méthode A ou videz le cache (voir §3).
<code>Authentication failed</code> avec demande d'username/password	Token mal collé ou expiré. Régénérez-en un nouveau.
« Support for password authentication was removed »	Vous saisissez votre mot de passe GitHub. Utilisez le token à la place.
Token perdu	Pas de récupération possible. Developer settings → Tokens → supprimez l'ancien et créez-en un nouveau.
Le token fonctionne mais accès refusé sur un dépôt	Le scope <code>repo</code> n'a pas été coché lors de la création.
<code>remote origin already exists</code>	Utilisez <code>git remote set-url origin <URL></code> au lieu de <code>git remote add origin <URL></code> .

6. Bonnes pratiques

- **Ne partagez jamais votre token.** C'est l'équivalent de votre mot de passe.
- **Ne le commitiez jamais** dans un fichier de code. Si cela arrive, GitHub le détecte et le révoque automatiquement.
- **Un token par usage** : un pour le cours, un pour vos projets perso, etc.
- **Renouvelez-le** quand il expire — on vous préviendra par e-mail.
- Pour un usage **professionnel ou long terme**, préférez la **méthode B + credential helper** plutôt que la méthode A (token en clair dans le projet).